



Improving UI responsiveness in Android by restructured rendering

Mingsong Lv^{a,b}, Tao Hu^c, Menglong Cui^b, Tao Yang^d, Yiyang Zhou^d, Qingxu Deng^a, Nan Guan^c

^a Northeastern University, China

^b The Hong Kong Polytechnic University, Hong Kong

^c City University of Hong Kong, Hong Kong

^d HUAWEI Technologies Company, China

ARTICLE INFO

Keywords:

Mobile operating system
UI responsiveness
UI rendering

ABSTRACT

Mobile operating systems, such as Android, are increasingly used across diverse applications, where ensuring high responsiveness to user interactions is critical, particularly in mission-critical and real-time scenarios. Mobile operating systems typically process user interaction events and UI rendering on the same thread, commonly referred to as the main thread of a mobile application. As a result, user interaction handling can face significant delays when blocked by overloaded UI rendering tasks, compromising responsiveness. Existing mobile operating systems lack effective mechanisms to mitigate this issue. This paper addresses the problem by restructuring the UI rendering workflow to improve responsiveness in the presence of heavy rendering workloads. Specifically, two techniques are proposed that are tailored to whether the event handling results require screen display. Experimental results demonstrate improvements in both average-case and worst-case response times of event handling, enhancing the UI responsiveness. Although the implementation focuses on Android, the proposed approaches are adaptable to other mobile operating systems with similar rendering architectures, such as iOS and HarmonyOS.

1. Introduction

Mobile operating systems (OSes), such as Android, are widely adopted not only in smartphones and tablets [1] but also in diverse domains such as home electronics [2,3], automotive systems [4,5], industrial control [6,7], and robotics [8,9], thanks to their mature ecosystems. Since users will frequently interact with these devices through a user interface (UI), it is critical for such devices to maintain high responsiveness to user interactions.

In a mobile application, user interaction (e.g., via a touch screen) and UI rendering are typically processed on the same thread, commonly called the *UI thread*. This thread executes tasks non-preemptively. In practice, UI rendering can experience transient overloads depending on the volume of content that needs to be rendered. If such an overload coincides with a UI event that requires processing, significant blocking time may be imposed on the event handling, thereby degrading UI responsiveness. Recent studies have highlighted this issue's impact on performance and safety, especially in mission-critical and real-time applications [10].

Consider the scenario of a smart TV application where users navigate through a media library using a remote controller to select a movie (Fig. 1(a)). Simultaneously, the screen scrolls to load and render new

content, creating a large rendering workload [11]. The heavy rendering task can block cursor movement, and users will experience a lag in their interaction with the TV.

Another example is drone control (Fig. 1(b)), where ground operators rely on a touchscreen UI to monitor and control the drone in real-time [12]. The interface simultaneously displays a live video feed from the drone's cameras and accepts control inputs, such as trajectory adjustment commands. For instance, if the operator issues a critical stop command while the system is rendering high-resolution video feeds, the processing of this command can be delayed, bringing safety risks to the drone. This example highlights the importance of maintaining high responsiveness in mission-critical applications.

While advanced CPUs in high-end devices can partially mitigate the above problems, many mobile devices, especially those in industrial control and robotics, typically adopt resource-constrained hardware due to cost considerations [13]. On such devices, the UI responsiveness problem can be severe. Unfortunately, there is a lack of effective mechanisms in mobile OSes to mitigate this issue.

This paper aims to *improve UI responsiveness on resource-constrained mobile devices*. Our approach restructures the UI rendering workflow (of the mobile OS) to minimize the user interaction handling delays caused

* Corresponding author at: Northeastern University, China.

E-mail address: lumingsong@cse.neu.edu.cn (M. Lv).



(a) A smart TV scenario where heavy UI rendering delays cursor movement, causing UI lag



(b) A drone control scenario where heavy UI rendering delays critical command processing [12]

Fig. 1. Examples of UI responsiveness problems.

by heavy UI rendering. Specifically, we propose two techniques depending on whether the handling results require display. For interaction handling that does not require display, we split overloaded rendering tasks into smaller, independently executable segments, allowing interleaved interaction handling. For those requiring display, we skip some non-essential rendering workload to pre-execute the essential rendering workload, decreasing its blocking to display interaction-handling results.

We propose an Android-based implementation designed to operate entirely at the user level without modifying the Android OS code. Experimental results demonstrate significant improvements in both average-case and worst-case response times of user interactions compared to existing Android mechanisms. While our solutions are demonstrated on Android, they can be adapted to other mobile operating systems with similar architectures, such as iOS [14] and HarmonyOS [15].

2. Background on UI systems

To facilitate understanding, we outline key aspects of a typical UI sub-system, taking Android as an example. Note that similar architectures also exist in iOS and HarmonyOS.

2.1. Android's UI thread and event handling

In Android, the graphical user interface (GUI) of an application is managed by a dedicated thread, known as the *UI thread* or the *main thread*. This thread is crucial for managing UI interactions and UI rendering [16]. The UI thread processes external events through a programming model that utilizes a handler mechanism, allowing event messages to be queued in the thread's message queue and processed sequentially in a non-preemptive manner. While it is theoretically possible to offload some UI tasks to secondary threads to alleviate the burden on the UI thread, this practice is rarely adopted due to the non-thread-safe nature of the rendering framework. Concurrent

modifications to UI elements by multiple threads can lead to race conditions, resulting in faulty behavior or application crashes. As the data objects for the UI are quite complex, a multithread implementation can be very risky. To mitigate these risks, Android mandates that all UI modifications must be confined to the UI thread.

2.2. The UI rendering workflow

A user interface comprises UI elements such as buttons and images organized in a hierarchical structure called *UI tree* (shown in Fig. 2(a)), manipulated by the UI rendering task. A complete rendering cycle involves computation on the UI thread, the render thread, and the GPU. The UI thread processes user inputs, updates UI elements, and compiles drawing instructions, which the render thread converts into GPU commands for rendering. The rendering result is a new image (also called a frame) to be displayed on the screen (Fig. 2(b)).

To ensure a smooth visual experience, a mobile OS tries to enforce a pre-defined frequency of UI rendering (60 times per second on most devices, and 120 times on high-end devices), typically referred to as *frame rate*. Each rendering cycle begins upon receiving a hardware VSYNC signal, and UI rendering is expected to be completed within the cycle. For example, on a 60 FPS system, the UI rendering is expected to complete within 16.6 ms upon a VSYNC triggering signal. Failure to meet this deadline results in dropped frames, leading to degraded visual lag.

A critical operation of UI rendering is *inflation* on the UI thread, where new UI elements are instantiated as Java objects. Inflation can be computationally expensive, a major source of long UI rendering delay, and often causes frame drops. To mitigate this problem, Android incorporates a prefetch mechanism for components like *RecyclerView* [17] that moves the inflations in a specific rendering cycle to the idle time slots in its previous rendering cycles.

2.3. The responsiveness challenge

The UI responsiveness is tightly coupled to the UI thread. Nonpreemptive task execution on the UI thread may cause heavy UI rendering to block UI event handling, producing noticeable delays that users experience. Inflation significantly contributes to slow UI rendering and introduces large UI event handling delays, particularly during system overload.

3. Task model and definitions

In this study, we explore two major tasks: *UI rendering tasks* and *UI event handling tasks*.

VSYNC signals trigger UI rendering at fixed intervals (T). Each VSYNC signal initiates a *rendering job* (j^r), which is executed across three components: the UI thread, the render thread, and the GPU. The execution times on these components are denoted as C_u , C_r , and C_g , respectively. Rendering jobs fall into two categories: *normal jobs* are those completed within the VSYNC period; *heavy jobs* contain extensive inflation operations, and their execution exceeds the VSYNC period. The execution times for a heavy job across the three components are represented as C_u^l , C_r^l , and C_g^l . Empirically, a normal job typically takes 2 ms~5 ms to finish, and a heavy job may take up to 140 ms to finish (around 9 VSYNC cycles under 60 FPS), according to our investigation into popular Android applications (details provided in Section 6.2). Inflation is performed per *ViewHolder*, a Java class that serves as a container of UI elements. Inflation time for a single *ViewHolder* is denoted as C_{vh} .

The second type of task, the *UI event handling task*, is sporadic, activated only upon the arrival of a user event at the UI thread. The time to handle a single UI event is denoted as C_e . The *response time* of a UI event e_i is defined as the interval between its arrival and the completion of its processing, classified as response time without display (R_i^n) or with display (R_i^d), depending on whether the processing result needs display. Fig. 3 illustrates these concepts.

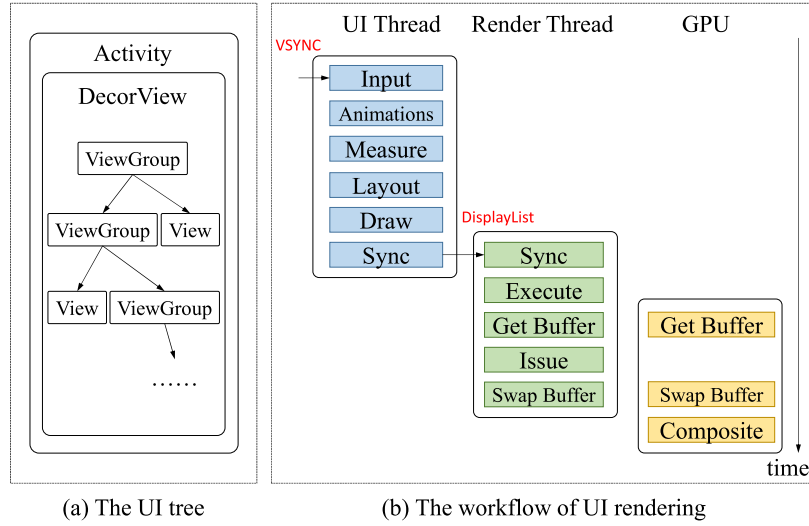


Fig. 2. The UI rendering workflow.

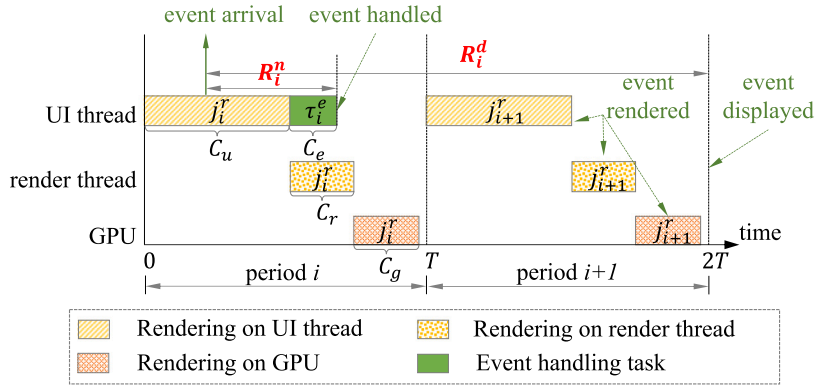


Fig. 3. Illustration of key concepts in UI rendering and event handling. The figure shows two rendering periods, with an event arriving during j_i^r in period i . If the event's result misses the i th rendering period, it is deferred to the next period ($i+1$), processed by j_{i+1}^r , causing a large response time R_i^d . If no display is required, the event is completed within period i , with a response time of R_i^n .

4. Reducing the response time without display

4.1. Solution overview

As shown in Fig. 4, when a UI event e_1 occurs during a long heavy job, it experiences significant blocking, leading to a high response time (R_1^n).

One classical solution is to reserve computing resources for critical tasks, such as UI event handling. However, due to the unpredictable nature of UI event arrivals, the strategy of reserving CPU time for unforeseen events is inefficient and even impractical. Another potential solution is to discard the heavy job to eliminate its blocking effect on event handling. This is also infeasible, as the inflation operations required for UI rendering within the heavy job must be performed.

A viable solution is to enable UI event handling to begin before the heavy job completes. To this end, we propose *job splitting*, which divides a heavy job into smaller segments. This segmentation allows the execution of event-handling tasks between the rendering segments, thereby reducing blocking time to event handling and improving responsiveness.

We provide a design by extending Android's prefetch mechanism, which preloads inflation operations in the idle periods preceding the heavy job. However, the standard prefetch mechanism has two fundamental limitations: first, it packs multiple inflations into a single handler executed non-preemptively: such a handler can still bring

extensive blocking to event handling; second, prefetching does not work upon unexpected heavy jobs, as it requires identifying heavy jobs before they occur.

To overcome these issues, our method modifies the prefetch mechanism to process one inflation at a time over multiple cycles if necessary. Additionally, when an unexpected heavy job arrives, we “postpone” it by modifying its arrival time to a future time point, allowing the prefetch mechanism to treat the heavy job as forthcoming and effectively preload inflations.¹ We refer to this technique as “Solution 1”, which is illustrated in Fig. 4.

4.2. The new UI rendering workflow

For the Android case, its UI rendering workflow is re-designed to incorporate Solution 1, as illustrated in Fig. 5. The original workflow components are shown in white blocks, while the newly introduced components for Solution 1 are highlighted in color. Typically, the

¹ A rendering job, upon its initiation, is linked to a message in the UI thread's message queue, marked with a timestamp dictating its execution timing. Although this timestamp is typically synchronized with the arrival of the corresponding VSYNC signal, we can adjust it to control the job's start time, for instance, scheduling it for a future point in time by delaying this timestamp.

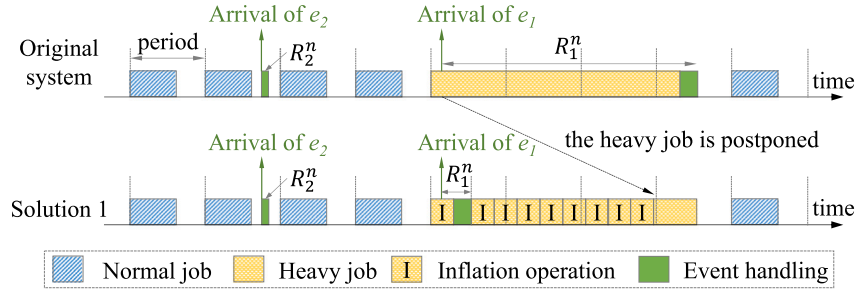


Fig. 4. Solution 1 reduces response time for events not requiring display. Event e_1 , arriving during a heavy job, executes only after the first inflation, experiencing less delay than the original system (R_1^n).

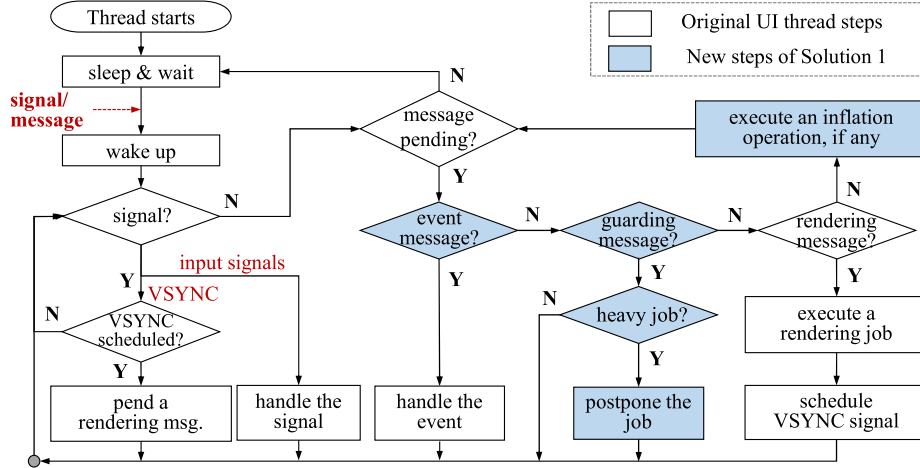


Fig. 5. The new rendering workflow under Solution 1. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

UI thread remains idle, waiting for incoming signals or messages. The receipt of a VSYNC signal prompts a new rendering message to the message queue. The UI thread iteratively processes messages in the queue sequentially, initiating screen rendering upon receiving a VSYNC message. The re-designed workflow has a new “guarding task”, triggered periodically by checking UI rendering messages, generated upon a VSYNC signal, and initiating a new rendering job. If a heavy job is identified, the guarding task applies Solution 1, dividing the job into smaller segments. This approach ensures that event-handling tasks can interleave with rendering segments of a heavy job.

The UI-thread scheduling semantics for Solution 1, covering signal/message handling and event responses, are illustrated in the pseudo-code of Algorithm 1. The principal processing stages (rectangles in Fig. 5) and decision points (decision diamonds in Fig. 5) are realized as the named functions in the algorithm.

4.3. Implementation of the key functions

This section gives the details on two key functions to enable Solution 1: to prefetch a single inflation operation and to postpone the heavy job.

4.3.1. Prefetching a single inflation operation

The standard Android prefetch mechanism starts by compiling a list of ViewHolder objects for inflation through the `onCreateViewHolder()` method, which is responsible for creating new ViewHolder instances. This mechanism aims to maximize the utilization of available free time slots by fitting multiple inflation operations into a single handler.

To extend this mechanism, we introduced a `doSplit()` function at the end of `onCreateViewHolder()`. After processing a single

inflation, this function clears the ViewHolder list and then rebuilds the list with the remaining inflations and triggers prefetching again. This iterative approach ensures that inflations are processed separately, minimizing their impact on event handling. Access to the related Java objects is achieved via Java reflection [18],² a mechanism that, at run time, inspects and manipulates classes and methods using metadata from the Java Virtual Machine.

4.3.2. Postponing the heavy job

Rendering jobs in Android are triggered by VSYNC messages in the UI thread’s message queue,³ with a `when` parameter indicating their scheduled start time. To postpone a heavy job, our approach modifies this parameter, deferring the job to a later time. The guarding task periodically inspects the message queue to identify heavy jobs by analyzing their inflation requirements. When a heavy job is detected, it is postponed conservatively, enabling effective inflation prefetching. To avoid unnecessary waiting or disruption, once the inflation operations are completed, the postponed heavy job is re-scheduled to execute immediately. This means if a heavy job is too big, it may cause frame losses both in the original system and in our system; but our system does not introduce extra waiting and thus no extra frame losses because no extra workload or waiting time is introduced.

² Java Reflection is a toolset for inspecting and manipulating classes and methods at runtime. It allows dynamic instantiation of objects and invocation of methods, providing access to class metadata within the Java Virtual Machine. Operations via the `java.lang.reflect` package unlock sophisticated runtime capabilities for JVM-based applications.

³ Following its execution, a job’s message is automatically removed from the message queue. The guarding task posts a new guarding message at the end of each cycle to ensure repetitive execution.

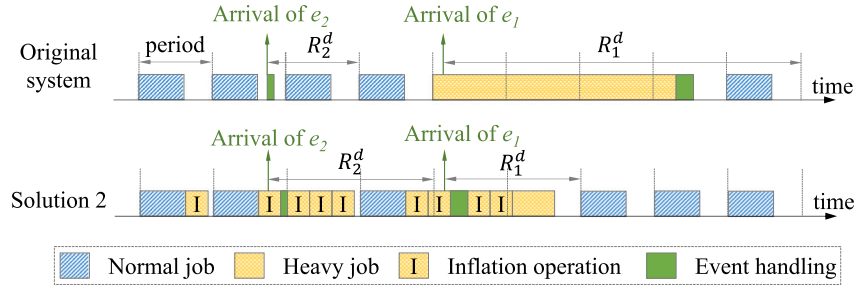


Fig. 6. In Solution 2, event e_1 arrives during a heavy job. e_1 would be delayed until the heavy job is completed (response time R_1^d). By skipping the 3rd normal job and executing inflations earlier, the heavy job finishes sooner, reducing the response time for e_1 . However, this causes a larger delay for e_2 , which can only be displayed in the 4th period.

Algorithm 1: Message handling on the UI thread (Solution 1)

```

Input: signal or message:  $\mathcal{M}$ 
Output: schedule of message handling

while TRUE do
    // suspend until a signal or message comes
    WaitForIncomingMessageOrSignal( $\mathcal{M}$ );
    // dealing with signals
    while IsSignalPending() do
        if IsSignal( $\mathcal{M}$ ) then
            if IsVsync( $\mathcal{M}$ ) AND IsVsyncScheduled()
            then
                PendEventMessage( $\mathcal{M}$ );
            else
                HandleSignal( $\mathcal{M}$ );
        // dealing with events
        while IsEventPending() do
            if IsNormalEvent( $\mathcal{M}$ ) then
                HandleEvent( $\mathcal{M}$ );
            else if IsGuardingMsg( $\mathcal{M}$ ) then
                if IsHeavyJob( $J$ ) then
                    Postpone( $J$ );
            else if IsRenderingMsg( $\mathcal{M}$ ) then
                ExecuteRendering( $J$ );
                ScheduleVsync();
            else
                ExecuteInflationIfAny();

```

Effective prediction of heavy jobs and balanced job-skipping management are crucial to handle this trade-off. The technical details of these components are described in Sections 5.3 and 5.4.

5.2. The new UI rendering workflow

The new workflow for Solution 2, illustrated in Fig. 7, integrates heavy job prediction and job-skipping mechanisms into the rendering pipeline. The guarding task plays a central role by manipulating VSYNC messages to manage job execution timing. Heavy job prediction works around UI parameters to anticipate upcoming heavy jobs. Based on these results, normal rendering jobs are selectively skipped according to a predefined plan, minimizing the impact on users' visual experience. After each heavy job is completed, the prediction mechanism re-calibrates to identify the next potential heavy job, ensuring continuous optimization.

The scheduling semantics for Solution 2 are illustrated in the pseudo-code of Algorithm 2. The principal processing stages (rectangles in Fig. 7) and decision points (decision diamonds in Fig. 7) are realized as the named functions in the algorithm. Here, function DecisionMaking() refers to the decision process of normal job skipping.

5.3. Heavy job prediction

This study focuses on heavy UI rendering workloads caused by screen scrolling, a major contributor to slow rendering in mobile applications, as documented in official Android developer reports [11]. Empirical studies further reveal that screen scrolling significantly affects user experience, being one of the most frequently performed interactions [19,20]. Research also highlights that screen scrolling substantially increases CPU utilization and power consumption [21].

Predicting heavy jobs means estimating after how many frames new UI elements will enter the screen and trigger a heavy job with many inflations. Accurately predicting a future heavy job requires understanding the dynamics of UI element movement, particularly in scenarios involving scrolling where screen velocity and acceleration are key parameters for determining when new elements will appear. However, precise prediction is complicated by the Android system's variable acceleration during scrolling, aimed at ensuring a fluid user experience, and the lack of direct access to initial speed metrics. To address these issues, we utilize Java reflection to extract essential screen movement parameters, enabling us to anticipate the appearance of new UI elements and thereby predict the arrival of a heavy job.

We use Eq. (1) to compute the estimated frame count (M) preceding the upcoming heavy job, the underlying principles of which are depicted in Fig. 8. In the equation, R denotes the frame duration, for example, $R = 16.6$ ms for 60 FPS. T_{total} , D_{total} , D_p , and X respectively represent the total scroll time duration until stationary, the total scroll distance until stationary, the distance already covered by the scroll, and the spatial distance to the point where new UI elements necessitate

5. Reducing the response time with display

5.1. Solution overview

When UI-event-handling results require a display, splitting a heavy job (as in Solution 1) is insufficient, as the display can only be updated after the entire heavy job finishes. This limitation is illustrated in Fig. 6, where event e_1 incurs a high response time (R_1^d) in the original system.

To solve this, our approach tries to complete the heavy job sooner by selectively skipping less critical normal jobs preceding the heavy job to offer extra time for its early execution. As shown in Fig. 6, skipping the 3rd normal job allows the execution of two additional inflations in advance, enabling the heavy job to be completed earlier and thus reducing R_1^d . This method is referred to as “Solution 2”.

However, skipping normal jobs introduces trade-offs, as it can delay other events. For example, since the 3rd normal job is skipped, e_2 , which arrived in the 2nd period, can only be displayed by the rendering job of the 4th period, which increases its response time.

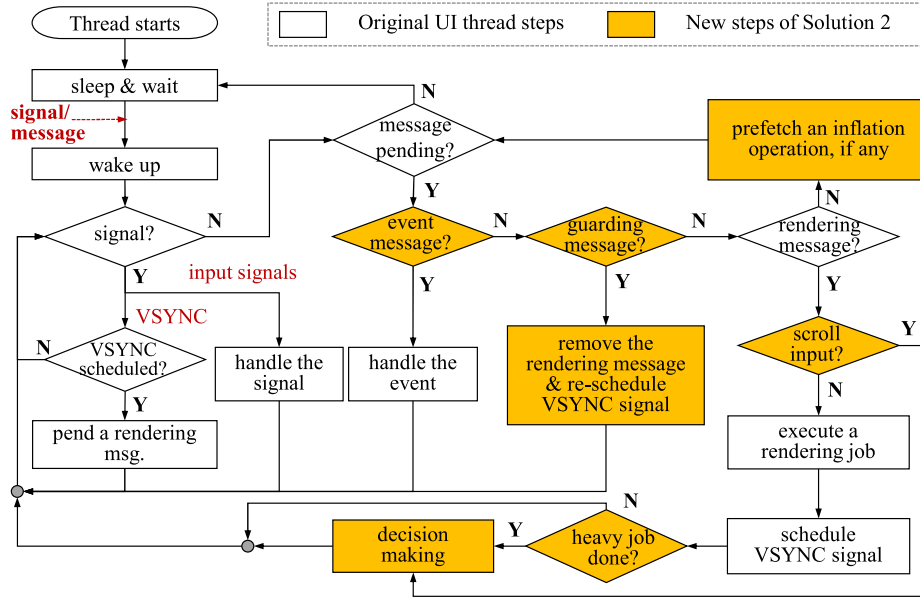


Fig. 7. The new rendering workflow under Solution 2.

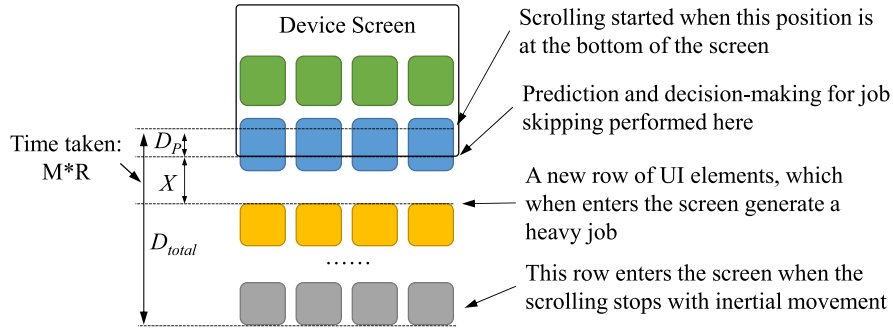


Fig. 8. Heavy job prediction based on user scroll interaction parameters.

inflation. The Android framework maintains 100 coefficients with no unit (K_1 to K_{100}), which are data points modeling the scroll distance-time relationship to realized variable speed scrolling for smoother experience. To determine the appropriate K_i to use, we need to first calculate $\frac{X+D_p}{D_{total}}$, and search through the 100 coefficients to find the one whose value is closest to $\frac{X+D_p}{D_{total}}$. The variables D_{total} , D_p , X , and K_i can be obtained from the Android system using Java reflection.

$$M = T_{total} \cdot \left(\frac{(\frac{X+D_p}{D_{total}} - K_i)}{100 \cdot (K_{i+1} - K_i)} + K_i \right) \cdot \frac{1}{R} \quad (1)$$

5.4. Job skipping

Job skipping can reduce response times for some UI events, such as e_1 in Fig. 6, but may also increase them for others, like e_2 . We apply a decision-making strategy to carefully balance these trade-offs. The goal is to minimize the maximum response time for events occurring within the prediction window by strategically skipping normal jobs, considering both cases like e_1 and e_2 .

For scenarios where job skipping could increase response times, we note that the extent of consecutive job omissions directly affects the potential maximum delay. Let i and j represent rendering periods where jobs are skipped, with $Y = j - i$ indicating the number of skipped jobs. The worst-case response time for a delayed event ($R_n(Y)$) occurs when the event just misses its rendering opportunity at the beginning

of period $i - 1$, necessitating a wait until period $j + 1$ for display. This duration is calculated as:

$$R_n(Y) = T \cdot (Y + 1 + \left\lceil \frac{C_u + C_r + C_g}{T} \right\rceil) \quad (2)$$

For events benefiting from job skipping, their response times are computed assuming N normal jobs precede the heavy job, and a maximum of Y consecutive skips are allowed. This permits skipping up to $\lfloor \frac{Y}{Y+1} \cdot N \rfloor$ jobs, thereby freeing up time for executing $\lfloor \frac{C_u}{C_{vh}} \rfloor \times \lfloor \frac{Y}{Y+1} \cdot N \rfloor$ inflation operations. The worst-case response time occurs when the event arrives just before the heavy job, with the processing result only becoming displayable after the heavy job's completion, calculated as:

$$R_h(Y, N) = T \cdot \left\lceil \frac{T + C_u^l - \lfloor \frac{C_u}{C_{vh}} \rfloor \min\{\lfloor \frac{Y}{Y+1} \cdot N \rfloor, N_l\} + C_r^l + C_g^l}{T} \right\rceil \quad (3)$$

Here, N_l is the number of inflations in the heavy job, and (Y, N) defines the job-skipping plan. Evaluating all (Y, N) combinations for an optimal plan is computationally impractical for runtime use. So, we collect execution times during application runs and use a background thread to compute a lookup table mapping (Y, N) pairs to their worst-case response times. Note that performance profiling and table generation can be performed automatically in the background without any user intervention. Once done, job skipping can be enabled. Regular updates of the performance data and the table can be performed to ensure accuracy over time.

Algorithm 2: Message handling on the UI thread (Solution 2)

Input: signal or message: $\mathcal{M}$
Output: schedule of message handling

```

while TRUE do
  // suspend until a signal or message comes
  WaitForIncomingMessageOrSignal( $\mathcal{M}$ );

  // dealing with signals
  while IsSignalPending() do
    if IsSignal( $\mathcal{M}$ ) then
      if IsVsync( $\mathcal{M}$ ) AND IsVsyncScheduled()
      then
        | PendEventMessage( $\mathcal{M}$ );
      else
        | HandleSignal( $\mathcal{M}$ );

  // dealing with events
  while IsEventPending() do
    if IsNormalEvent( $\mathcal{M}$ ) then
      | HandleEvent( $\mathcal{M}$ );
    else if IsGuardingMsg( $\mathcal{M}$ ) then
      | RemoveMsg( $\mathcal{M}$ );
      | ScheduleVsync();
    else if IsRenderingMsg( $\mathcal{M}$ ) then
      if NotScrollInput() then
        | ExecuteRendering( $J$ );
        | ScheduleVsync();
        if IsHeavyJobDone() then
          | DecisionMaking();
      else
        | DecisionMaking();
    else
      | ExecuteInflationIfAny();

```

5.5. Implementation of the key functions

In this subsection, we provide the technical details of implementing heavy job prediction and job skipping within the UI framework.

Heavy Job Prediction: The implementation of the prediction mechanism is integrated into `doFrame()` via the `doCallbacks()` interface, triggered by user inputs or heavy job completion (to predict the next heavy job). Job execution times are logged using our developed time-tracking APIs, which need to be integrated into mobile applications. The number of inflation operations in the heavy job (N_j) is quantified via the layout manager's `collectAdjacentPrefetchPositions()` method.

Job Skipping: For effective job skipping, accurately obtaining the execution times of rendering jobs is crucial. We incorporated time-tracking APIs that developers must integrate within their application code to log the start and end points of rendering jobs. Additionally, quantifying the inflation operations (N_j) involved in heavy jobs is critical due to their significant impact on rendering duration. We utilize the layout manager, which orchestrates the positioning and interrelations of UI elements, for this purpose. The `collectAdjacentPrefetchPositions()` method within the layout manager is used to determine the number of ViewHolders about to enter the viewing area, and Java reflection is employed to access this method and N_j . Given the rendering delay may vary, decision-making relies on average execution times derived from extensive application runtime data collected prior to deploying our strategy.

For the implementation of job skipping, consider a scenario where rendering jobs are skipped from period i to j . The “guarding task” is employed to monitor the message queue. When a rendering message in

the skipping interval is enqueued upon a VSYNC signal, the guarding task promptly removes it. To ensure rendering will resume in period $(j+1)$, the guarding task is aligned preemptively with the VSYNC signal for the $(j+1)$ th frame, re-inserting a rendering message into the message queue. This manipulation of the message queue is achieved through Java reflection. Fig. 9 illustrates when heavy job prediction, selective job skipping, and the preemptive execution of inflation operations are performed.

6. Experiments & evaluation**6.1. Experimental setup**

Hardware and software platform: Experiments were conducted on a Google Pixel 4 smartphone with a Qualcomm Snapdragon 855 processor with eight Kryo 485 cores (clocked at 1.4 GHz for our tests) and an Adreno 640 GPU. The device's OLED screen has a resolution of 1080×2280 pixels, supports a 24-bit color depth, and operates at a refresh rate of 60 FPS. The use of Pixel 4, a model behind the current high-end smartphones, is intentional to mirror the resource constraints typically found in industrial devices. Despite using Android 10, an older Android version, the core UI rendering architecture relevant to our study remains consistent with newer Android versions.

Benchmark programs: Our techniques do not require modifications to the Android system itself. However, user applications must incorporate some new APIs provided by our techniques to collect execution information. Therefore, conducting experiments on commercial apps was not feasible. Instead, we developed a synthetic application featuring scrollable movie image rows with adjustable UI element density to simulate varying workloads. Events were injected into the application by a separate thread. Event generation and dispatching were managed by a separate thread to allow precise measurement of event response times across various rendering load scenarios.

Compared approaches and evaluation metrics: We compared several configurations to assess their effectiveness in managing external events:

- **Baseline Android UI Rendering:** The standard UI rendering process in Android, without any specific adjustments for heavy rendering.
- **Android UI Rendering with Prefetch:** Enhances the baseline by integrating Android's built-in prefetching mechanism to preload inflation operations.
- **Proposed Solutions:** Both Solution 1 and Solution 2 proposed.

Evaluation Targets: Our experimental analysis focused on measuring UI event response times under varies rendering workloads. We also evaluated the accuracy of our heavy job prediction mechanism within scrolling contexts and quantified any additional overhead introduced by our techniques, providing a comprehensive assessment of their practical implications.

6.2. Exploring real-world workload conditions

To accurately simulate light and heavy workload scenarios in our custom Android application, we conducted a detailed analysis of the typical performance characteristics observed in real-world Android apps. Our goal was to understand the range of time delays experienced by the UI thread during both standard and intensive rendering tasks. This involved observing a variety of commonly used applications under typical usage conditions, such as scrolling through different types of content, for 20 s each session.

To quantify these workloads, we employed specific Android APIs to monitor and log rendering delays on the UI thread. The results of our measurements are depicted in Fig. 10, where the horizontal axis represents time and the vertical axis indicates the delay incurred to produce a new frame. Our experimental findings highlighted significant performance variability across different applications. For example, applications like BigoLive and YouTube consistently generated

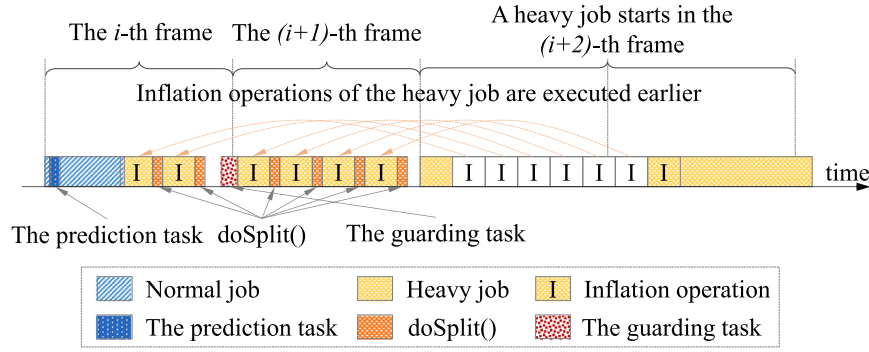


Fig. 9. Timeline showing heavy job prediction, job skipping, and inflation advancement. A heavy job is predicted to occur in the $(i+2)$ th frame, with prediction done in the i th frame and a normal job skipped in the $(i+1)$ th frame.

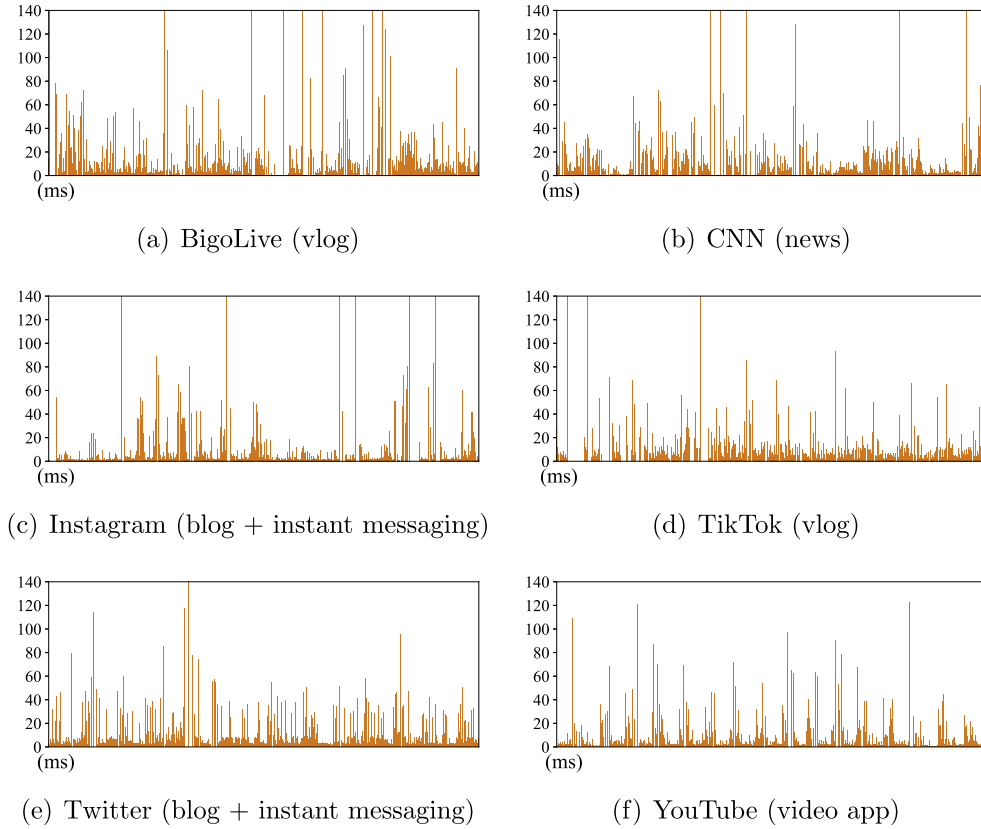


Fig. 10. Empirical study on the workload of rendering on the UI thread for six popular Android applications. The x-axis denotes time, and the y-axis denotes the delay on the UI thread for all the rendered frames during the experiment window.

heavy workloads, whereas applications such as Instagram and Twitter triggered such conditions less frequently.

Under *light workload conditions*, normal rendering jobs typically resulted in rendering delays ranging from 2 ms to 5 ms (e.g., Instagram). Conversely, in *heavy workload conditions*, rendering tasks exhibited delays between 50 ms and 70 ms (e.g., TikTok). In extreme cases, delays for normal rendering jobs may reach up to 40 ms, while those for heavy jobs are up to about 140 ms.

Based on this exploration, we configured light and heavy workloads in our test app, with normal and heavy task delays averaging 3 ms/70 ms (light workload conditions) and 15 ms/90 ms (heavy workload conditions), respectively. Events were processed in 1 ms, with each trial spanning 20 s, during which approximately 30 heavy rendering jobs were observed. Each experiment comprised 2000 trials with evenly distributed event triggers.

6.3. Evaluation of Solution 1

This set of experiments aimed to assess the effectiveness of Solution 1 for events not requiring display, specifically job splitting in reducing event response times. The response time data for the three approaches under comparison are illustrated in Fig. 11.

Results under heavy workloads in Fig. 11(a) show significant delays for the baseline Android system, with peak response times (WCRT) exceeding 100 ms. Android's prefetch provided minor improvements by advancing some inflation operations but was limited by insufficient free slots, shown in Fig. 11(c). However, the effectiveness of this approach was limited by the sparse availability of free time slots in a heavy workload environment. Our Solution 1, on the other hand, notably reduced response time peaks, shown in Fig. 11(e). By splitting all inflation operations, even events coinciding with a heavy job, they were

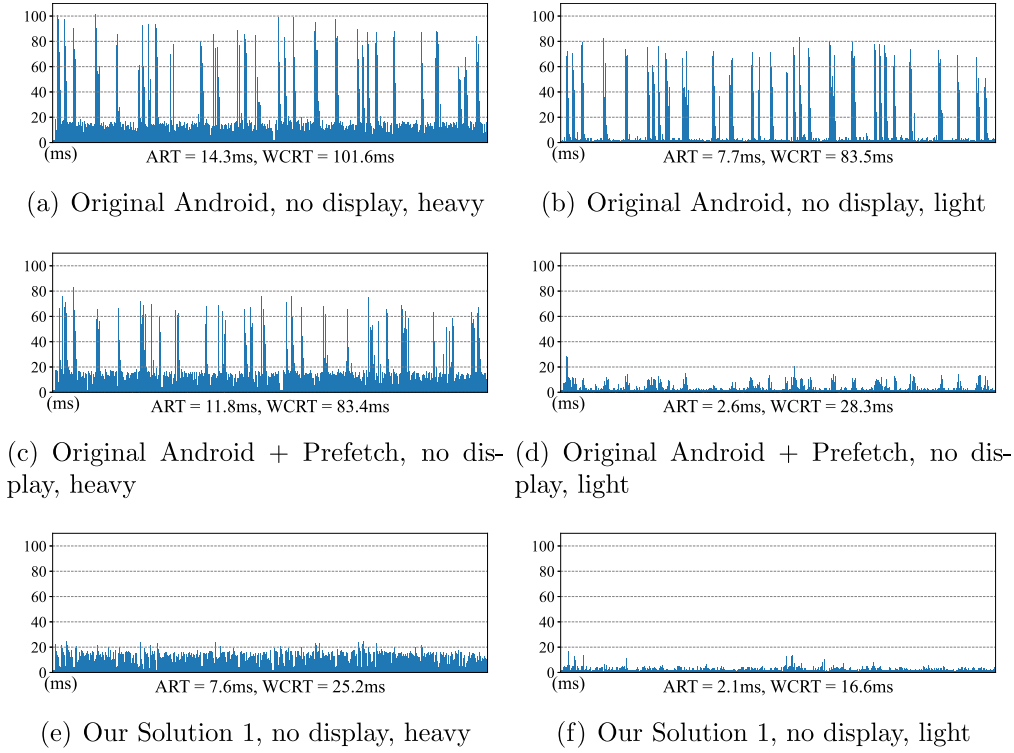


Fig. 11. Experimental results on the events' response time with no display under both heavy and light workload scenarios. The average and maximal observed response times (ART and WCRT) are listed under each sub-figure.

only blocked by at most one inflation operation, leading to a reduction of ART and WCRT by 45% and 75.8%, respectively, compared to the baseline system.

Fig. 11(b, d, f) explore the light workload scenario, where the delay for normal jobs is typically small, leaving ample free time within the frame periods. The baseline Android system continues to show the worst performance, particularly when an event coincides with a heavy job. In contrast, the prefetch mechanism effectively utilized the abundant free slots within the frames of normal jobs for prefetched inflation operations, resulting in shorter blocking times for events. Our approach surpassed Android's prefetch by ensuring each inflation operation was separated, reducing the blocking experienced by events even when an event coincided with a prefetch job.

6.4. Evaluation of Solution 2

This second set of experiments was designed to demonstrate the effectiveness of Solution 2 in minimizing response times for events where display of results is required. We applied the same experimental setup as described above. To accurately assess response times for display-required events, it was crucial to capture the precise moment when an event's handling result is rendered on the screen. Notably, after an event is processed, it is queued in the rendering pipeline and displayed in the next un-skipped frame. The response time also includes the time spent on the render thread and the GPU during the rendering of the frame. We utilized the FrameStats metrics provided by the Android system to measure the total rendering duration. The results are shown in Fig. 12.

In heavy workload conditions, the baseline Android system exhibited significantly prolonged event response times (Fig. 12(a)). The prefetching mechanism provided marginal reductions in these delays (Fig. 12(c)), as events coinciding with heavy jobs still faced lengthy waiting times for their results to be displayed. Conversely, Solution 2 effectively reduced these delays by skipping less critical frames, thus freeing up time slots for the early execution of inflation operations

associated with heavy jobs. This strategy significantly shortened the duration of heavy jobs and reduced event blocking time (Fig. 12(e)), achieving a 57% reduction in WCRT from 132.7 ms to 76 ms. The ART changed only marginally because even if some normal jobs were skipped, the new free time slots were occupied by inflation operations from the heavy jobs, which did not notably reduce the response time for events arriving during normal jobs.

Under light workloads (Fig. 12(b, d, f)), the prefetch mechanism performed better due to abundant free slots in normal frames, which facilitated efficient handling of prefetched inflation operations. In these scenarios, Solution 2 performed comparably to the prefetch mechanism, taking advantage of the ample free time slots to efficiently manage inflation operations.

To further evaluate Solution 2, we investigated the accuracy of our job-skipping prediction mechanism and quantified the extent of normal frame skipping needed to facilitate heavy rendering tasks. We simulated realistic user interactions by generating scrolling input for each trial, with each experiment lasting about 10 s and involving 14 heavy jobs.

Throughout these trials, we monitored the number of frames preceding each heavy job, distinguishing between skipped frames (SF) and frames that retained rendering activity (TF), with PF representing the predicted number of normal frames before a heavy job. Ideally, $PF = SF + TF$. The analysis, presented in Fig. 13, shows PF closely aligning with the sum of SF and TF, demonstrating the precision of our predictions. This precision is attributed to our prediction relying on key parameters on screen movements retrieved from the Android system. Typically, there were 13 to 14 normal frames before a heavy job, with about 4 to 5 frames skipped to accommodate inflation operations. Our job skipping strategy ensured that these skipped frames were distributed throughout the 13 to 14 normal frames, avoiding consecutive skipping and maintaining a good user experience.

6.5. Overhead evaluation

We measured the overhead of our proposed approaches with time-tracking functions surrounding our introduced components.

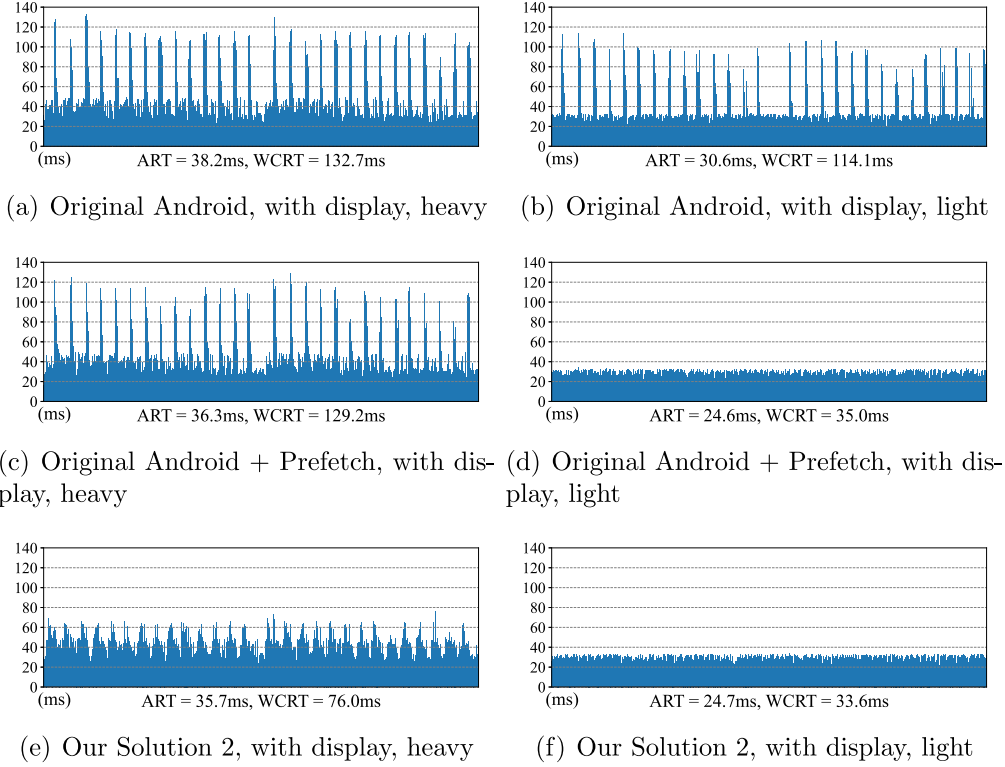


Fig. 12. Experimental results on the events' response time with display, under both heavy and light workload conditions. The average and maximal observed response times (ART and WCRT) are listed under each sub-figure.

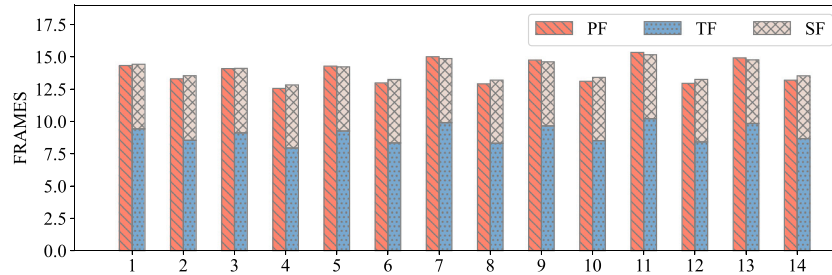


Fig. 13. Analysis of frame skipping under heavy workload scenarios, with information on heavy job prediction and normal job skipping.

Prediction Overhead: Central to Solution 2 is a mechanism for predicting the occurrence of heavy jobs. This prediction is seamlessly integrated within the `doFrame()` function and occurs once per specific frame. Since the job-skipping plan is predetermined offline, the run-time overhead for this prediction process is negligible, amounting to only 5 μ s.

Guarding Task Overhead: The job-skipping strategy includes a guarding task that monitors incoming VSYNC messages on the UI thread. The operational overhead for the guarding task is relatively modest, consuming about 1.2 ms per cycle. This time includes 160 μ s dedicated to managing the message queue and adjusting VSYNC signals. A significant portion of this overhead stems from using Java reflection to access system-level information. This overhead can be reduced with an alternative implementation of our approaches within the Android framework.

Splitting Overhead: Our approach enhances the existing prefetch mechanism by incorporating a `doSplit()` function at the end of the

Android method `onCreateViewHolder()`. The overhead for this operation is twofold: the execution of `doSplit()` averages 155 μ s, and the reconstruction of the task list for pending inflation operations takes approximately 0.5 ms.

Additionally, gathering average execution time data for rendering tasks incurs a one-time overhead, requiring the application to run for a brief period initially. This preliminary step is crucial for setting up our solutions and does not impact runtime performance. Users or developers can update these metrics as needed by re-running this sampling process at an appropriate frequency during runtime, ensuring the system's adaptability to changing performance characteristics.

7. Discussions

Impact on User Experience. The proposed solutions also enhance the user experience. As shown in Figs. 11 and 12, our methods reduce

perceived latency and stabilize frame rates, providing a smoother and more responsive interaction. Regarding the negative side of frame skipping, we observed that fewer than half of the total normal frames were skipped to accommodate heavy tasks, with consecutive skipping avoided. User experience studies indicate that isolated frame drops are less disruptive than consecutive ones [22]. Our skipping mechanism strategically ensures that user-perceived smoothness is maintained.

Adaptability to Other Mobile OSes. While our implementation is based on Android, the underlying principles apply to other mobile operating systems like iOS [14] and OpenHarmony [15], which feature similar UI rendering frameworks. In iOS, UI rendering and event handling are managed on the `MainThread`, analogous to Android's UI thread, and driven by `VSYNC` signals. Capabilities like `UIApplicationHandleEventQueue` for event handling and `UICollectionViewDataSourcePrefetching` for prefetching are comparable to Android's mechanisms. OpenHarmony, a newer mobile OS, also shares a similar rendering architecture, indicating our solutions could be adapted.

Portability across SoCs. Our approaches are designed to be chipset-agnostic. It depends on stable UI-pipeline primitives (e.g., input delivery, `doFrame()`, `measure/layout/inflation`, `prefetch`) and uses the frame period together with online timing signals as inputs, rather than on vendor-specific features or hard-coded latencies. Thresholds are normalized to T and adapt to DVFS/frequency changes, so absolute latencies may shift across mobile CPUs, but the decision rules remain unchanged. The GPU pipeline is treated as an external resource: when fence waits or queue depth suggest GPU pressure, the policy de-emphasizes CPU run-ahead to avoid wasted cycles, independent of the underlying GPU architecture. Taken together, the mechanisms are orthogonal to specific SoC vendors.

Generalization to More Application Scenarios. In this study, we exemplified our approach with screen scrolling scenarios, a choice supported by compelling reasons. Screen scrolling is a well-documented source of slow rendering issues in mobile applications, as highlighted in official Android developer reports [11]. Empirical data from extensive user interaction studies shows that screen scrolling is among the most frequently performed actions, significantly impacting user experience [20]. For example, authors collected over 82 million user interaction events, with screen scrolling comprising 30 million, making it the most common type of interaction. Further investigations into performance bugs reveal that lengthy operations during scrolling can severely impair the responsiveness and smoothness of applications [19]. Additionally, studies on optimizing energy consumption in Android applications indicate that scrolling is a major contributor to CPU utilization and power consumption, with some instances accounting for up to 59.8% of total energy usage [21].

While scrolling is a critical UI interaction scenario, the principles derived from our work are adaptable to other contexts as well. For instance, adapting our methods to scenarios such as switching between applications, which can trigger sudden spikes in rendering workload, might require new heavy job prediction methods. Insights from recent research on predicting user behavior [23–26] could be leveraged, based on the analysis of users' usage habits. Implementing background tasks for real-time user data collection, combined with analytical models or deep neural networks (DNNs), may enable the prediction of event triggers in a broader context.

Alternative Implementation Approaches. While our current prototypes are realized at the application (user) level to avoid modifying Android system code, some foundational enablers, such as time tracking and logging, still require additional implementation. These enablers can be provided either in the OS or with minor application-side changes. In this paper we adopt the latter choice: the mobile OS remains unchanged, and applications require only minor modifications. Importantly, the techniques themselves (job splitting, heavy-job prediction, and job skipping) do not depend on application-specific semantics, so they can also be integrated directly into the mobile OS UI architecture

to benefit legacy applications without altering their code. For example, in Android, job splitting and job skipping could be incorporated into the system prefetch task, while heavy-job prediction could be integrated into `doFrame()`. An OS-level realization may further reduce overhead and provide broader, system-wide improvements. Overall, selecting between OS-level and app-level realizations is a deployment trade-off rather than a limitation of our techniques; it is a decision to be made prior to product integration.

Relationship with Classical Soft Real-Time Techniques. Classical soft real-time systems [27] require timely responses to be prioritized, but occasional deadline misses are tolerable. A main design target is to ensure graceful degradation of QoS (such as response times of tasks) in overloaded situations. Common solution techniques include resource reservation, job skipping, period adaptation, etc. These approaches often work for simpler control real-time tasks with relatively more predictable execution times and arrival patterns. In summary, classical soft real-time techniques focus on re-arranging the execution sequence of different jobs to improve QoS. In contrast, the highlight of our proposed approaches is mainly re-shaping the critical jobs (i.e., the heavy jobs) to improve QoS with domain-specific techniques. Of course, our overall solution also applies techniques similar to those in soft real-time domain, such as job skipping (for normal jobs).

8. Related work

UI responsiveness, as a crucial performance metric of mobile systems, has attracted significant research attention in recent years [28]. Several studies have focused on performance analysis to identify performance bottlenecks within Android applications [29–35]. These studies aim to identify causes of sluggish UI performance, providing insights for optimizing mobile OSes and applications.

Some research interprets UI responsiveness as the frame rate a system can achieve since a higher frame rate generally results in smoother displays. For instance, Li et al. introduced techniques to ensure high frame rate by reducing the UI rendering workload, through reusing rendering results for unchanged Views across adjacent frames [36,37]. While such approaches enhance frame rates, their focus lies in reducing GPU workload rather than CPU competition between UI rendering and event handling. Callan and Petke explored code modifications to boost Android application frame rates by automatically identifying software components leading to the highest frame rate [38]. Compared to related methods that tried to reduce the workload of a UI rendering task, our work restructures the rendering task and appropriately schedules the rendering sub-tasks and the event handling task to reduce the response time of event handling.

Other studies explore UI responsiveness in terms of transition times between different Android activities [39], which does not directly relate to our work. Yu et al. investigated adaptive frame rate adjustments in smartphone applications, focusing on reducing power consumption rather than enhancing event responsiveness [21]. Their approach to managing rendering workloads differs significantly from ours. Dietrich et al. proposed RT.js as a new execution model for JavaScript programs, providing preemptive task scheduling [40]. But this cannot be directly applied in UI rendering, as preemption in mobile OSes' UI threads is generally disallowed due to risks of race conditions.

9. Conclusion

Mobile operating systems, such as Android, iOS, and HarmonyOS, encounter challenges in maintaining UI responsiveness due to competition for CPU resources between UI event handling and UI rendering, particularly on resource-constrained devices. This paper addresses these challenges by proposing a system that restructures the existing UI rendering workflow to improve UI responsiveness. Implemented and validated with Android, the proposed methods demonstrate marked improvements in UI responsiveness across diverse workload scenarios.

CRediT authorship contribution statement

Mingsong Lv: Writing – review & editing, Writing – original draft, Supervision, Project administration, Methodology, Investigation. **Tao Hu:** Writing – original draft, Validation, Software, Investigation. **Menglong Cui:** Validation, Software, Investigation. **Tao Yang:** Investigation, Feasibility analysis, Evaluation, Supervision, Resources. **Yiyang Zhou:** Investigation, Feasibility analysis, Evaluation, Supervision, Resources. **Qingxu Deng:** Methodology, Funding acquisition. **Nan Guan:** Supervision, Project administration, Methodology, Investigation, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is partially sponsored by Huawei Innovation Research Program “Energy-Efficiency Aware Collaborative Computing for Heterogeneous OS Platforms” and the Hong Kong GRF (Grant Number: 15206221, 11209122 and 11208522).

Data availability

No data was used for the research described in the article.

References

- [1] Android statistics, 2025, <https://www.businessofapps.com/data/android-statistics>. (Accessed 10 August 2025).
- [2] Android TV, 2025, <https://www.android.com/tv/>. (Accessed 10 August 2025).
- [3] G.-M. Sung, H.-K. Wang, W.-T. Su, Smart home care system with fall detection based on the android platform, in: 2020 IEEE International Conference on Systems, Man, and Cybernetics, SMC, 2020, pp. 3886–3890.
- [4] S. Sinha, A. Golchin, C. Einstein, R. West, A paravirtualized android for next generation interactive automotive systems, in: The 21st International Workshop on Mobile Computing Systems and Applications, 2020, pp. 50–55.
- [5] H. Li, H. Zhang, L. Huang, K. Shao, G. Yang, N. Zheng, X. Jiang, Android evaluation methodology in automotive electronics, in: 2018 IEEE International Conference on Internet of Things (IThings) and Co-Located Conferences, 2018, pp. 695–699.
- [6] A. Drumea, Control of industrial systems using android-based devices, in: Proceedings of the 36th International Spring Seminar on Electronics Technology, 2013, pp. 405–408.
- [7] M. Wilcox, J. Vossaert, V. Raes, V. Naessens, Using android devices as mobile extensible HMIs, in: 2018 Fifth International Conference on Internet of Things: Systems, Management and Security, 2018, pp. 222–227.
- [8] N. Rottmann, N. Studt, F. Ernst, E. Rueckert, Ros-mobile: An android application for the robot operating system, 2020, CoRR, abs/2011.02781. Available: <https://arxiv.org/abs/2011.02781>.
- [9] S. Gobel, R. Jube, S.-L. Raesch, A. Zundorf, Using the android platform to control robots, in: The 2nd International Conference on Robotics in Education, 2011, pp. 135–142.
- [10] J. Fu, Y. Wang, Y. Zhou, X. Wang, How resource utilization influences UI responsiveness of android software, Inf. Softw. Technol. 141 (2022) 106728, <http://dx.doi.org/10.1016/j.infsof.2021.106728>.
- [11] Slow rendering, 2024, <https://developer.android.com/topic/performance/vitals/render>. (Accessed 10 August 2025).
- [12] Drone remote control product page, 2024, <https://play.google.com/store/apps/details?id=com.fly.smart.drone>. (Accessed 15 March 2024).
- [13] Android 10 tablet p8 by comnav technology ltd, 2025, <https://www.directindustry.com/prod/comnav-technology-ltd/product-234330-2365135.html>. (Accessed 10 August 2025).
- [14] Apple developer documentation, 2025, <https://developer.apple.com/documentation/>. (Accessed 10 August 2025).
- [15] Harmonyos official website, 2025, <https://www.harmonyos.com/en/>. (Accessed 10 August 2025).
- [16] Processes and threads overview, 2024, <https://developer.android.com/guide/components/processes-and-threads>. (Accessed 10 August 2025).
- [17] Recyclerview prefetch, 2016, <https://medium.com/google-developers/recyclerview-prefetch-c2f269075710>. (Accessed 10 August 2025).
- [18] Using java reflection, 1998, <https://www.oracle.com/technical-resources/articles/java/javareflection.html>. (Accessed 10 August 2025).
- [19] Y. Liu, C. Xu, S. Cheung, Characterizing and detecting performance bugs for smartphone applications, in: The 36th International Conference on Software Engineering, ICSE, 2014, pp. 1013–1024.
- [20] B. Friedrichs, L.D. Turner, S.M. Allen, Discovering types of smartphone usage sessions from user-app interactions, in: IEEE International Conference on Pervasive Computing and Communications Workshops and Other Affiliated Events, PerCom Workshops, 2021, pp. 459–464.
- [21] J. Yu, H. Han, H. Zhu, Y. Chen, J. Yang, Y. Zhu, G. Xue, M. Li, Sensing human-screen interaction for energy-efficient frame rate adaptation on smartphones, IEEE Trans. Mob. Comput. 14 (8) (2014) 1698–1711.
- [22] Z. Tan, N. Dai, Y. Su, X. Gao, X. Liu, How does frame-loss affect users' perception of smoothness? CCF Trans. Perv. Comput. Interact. 3 (2) (2021) 199–221.
- [23] R. Baeza-Yates, D. Jiang, F. Silvestri, B. Harrison, Predicting the next app that you are going to use, in: The 8th ACM International Conference on Web Search and Data Mining, WSDM 2015, 2015, pp. 285–294.
- [24] C. Degott, N.P. Borges Jr., A. Zeller, Learning user interface element interactions, in: The 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA, 2019, pp. 296–306.
- [25] W. Cao, C. Xia, S.T. Peddinti, D. Lie, N. Taft, L.M. Austin, A large scale study of user behavior, expectations and engagement with android permissions, in: 30th USENIX Security Symposium, USENIX Security, 2021, pp. 803–820.
- [26] Z. Liu, C. Chen, J. Wang, Y. Huang, J. Hu, Q. Wang, Guided bug crush: Assist manual GUI testing of android apps via hint moves, in: Conference on Human Factors in Computing Systems, 2022, pp. 557:1–557:14.
- [27] G. Buttazzo, G. Lipari, L. Abeni, M. Caccamo, Soft Real-Time Systems: Predictability vs. Efficiency, Plenum Publishing Co., 2005.
- [28] M. Hort, M. Kechagia, F. Sarro, M. Harman, A survey of performance optimization for mobile applications, IEEE Trans. Softw. Eng. 48 (8) (2022) 2879–2904.
- [29] D.K. Hong, A. Nikraves, Z.M. Mao, M. Ketkar, M. Kishinevsky, Perfprobe: A systematic, cross-layer performance diagnosis framework for mobile platforms, in: IEEE/ACM 6th International Conference on Mobile Software Engineering and Systems, MOBILESoft, 2019, pp. 50–61.
- [30] Y. Kwon, S. Lee, H. Yi, D. Kwon, S. Yang, B.-G. Chun, L. Huang, P. Maniatis, M. Naik, Y. Paek, Mantis: Automatic performance prediction for smartphone applications, in: USENIX Annual Technical Conference, USENIX ATC 13, 2013, pp. 297–308.
- [31] Y. Kang, Y. Zhou, M. Gao, Y. Sun, M.R. Lyu, Experience report: Detecting poor-responsive ui in android applications, in: 27th IEEE International Symposium on Software Reliability Engineering, ISSRE, 2016, pp. 490–501.
- [32] Z. Lei, W. Zhao, Z. Ding, M. Xia, Z. Qi, Appspin: Reconfiguration-based responsiveness testing and diagnosing for android apps, Autom. Softw. Eng. 29 (2) (2022) 47.
- [33] W. Zhao, Z. Ding, M. Xia, Z. Qi, Systematically testing and diagnosing responsiveness for android apps, in: IEEE International Conference on Software Maintenance and Evolution, ICSME, 2019, pp. 449–453.
- [34] Y. Wang, A. Rountev, Profiling the responsiveness of android applications via automated resource amplification, in: The International Conference on Mobile Software Engineering and Systems, MOBILESoft 2016, 2016, pp. 48–58.
- [35] T. Ongkosit, S. Takada, Responsiveness analysis tool for android application, in: The 2nd International Workshop on Software Development Lifecycle for Mobile, 2014, pp. 1–4.
- [36] X. Li, G. Li, X. Cui, Retriple: Reduction of redundant rendering on android devices for performance and energy optimizations, in: 57th ACM/IEEE Design Automation Conference, DAC, 2020, pp. 1–6.
- [37] X. Li, G. Li, Hb-retriplace: Mobile rendering optimization based on efficient history reusing, J. Syst. Archit. 129 (2022) 102627.
- [38] J. Callan, J. Petke, Improving android app responsiveness through automated frame rate reduction, in: International Symposium on Search Based Software Engineering, 2021, pp. 136–150.
- [39] J. Callan, J. Petke, Improving responsiveness of android activity navigation via genetic improvement, in: The ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings, 2022, pp. 356–357.
- [40] C. Dietrich, S. Naumann, R. Thrift, D. Lohmann, Rts.js: Practical real-time scheduling for web applications, in: 2019 IEEE Real-Time Systems Symposium, RTSS, 2019, pp. 69–79.



Mingsong Lv received the B.S., M.S., and Ph.D. degrees in Computer Science from Northeastern University, China, in 2002, 2005, and 2010, respectively. He is an Associate Professor at Northeastern University, China. His research interests include real-time systems, the Internet of Things, and mobile operating systems. He received Best Paper Awards at RTSS 2022 and DATE 2013, and Best Paper Nominations at DATE 2023 and RTSS 2010.



Tao Hu received the B.S. and M.S. in Internet of Things Engineering from Northeastern University, China, in 2019. He is currently a Ph.D. candidate at City University of Hong Kong. His research interests include mobile operating systems, real-time systems, and AI agents.



Yiyang Zhou received his M.S. degree from Department of Computer Science and Technology, Nanjing University. He is currently the Senior Engineer in Central Software Institute of Huawei Technologies Co., Ltd. His research interests include parallel-render, collaborative-compute, low-power and virtualization in Terminal OS.



Menglong Cui received the M.S. degree in Software Engineering, Yunnan University, China. He was a Research Assistant at the Department of Computing, the Hong Kong Polytechnic University. His research interests include mobile operating systems and multi-device interconnection.



Qingxu Deng received the Ph.D. degree in computer science from Northeastern University, China, in 1997. He is a professor with the School of Computer Science and Engineering, Northeastern University, China. His research interests include cyber-physical systems, embedded systems, and real-time system.



Tao Yang received his Ph.D. degree from the Northeastern University, Shenyang, China. He is currently the Chief Architect of Terminal OS, Central Software Institute of Huawei Technologies Co., Ltd. His research interests include multiprocessor real-time scheduling and operating system architecture.



Nan Guan received the B.E. and M.S. degrees from Northeastern University, China, in 2003 and 2006, respectively, and the Ph.D. degree from Uppsala University, Sweden, in 2013. He is currently a professor with the Department of Computer Science, City University of Hong Kong. His research interests include LLM-aided system design, autonomous driving systems, real-time systems, and real-time robotic operating system (ROS).